Qwen

From Packet Capture to Peer-to-Peer: A Rust-Based Framework for Reviving Discontinued Multiplayer Games

The digital preservation of multiplayer video games presents a formidable challenge distinct from that of single-player titles. While static binary archives and emulators suffice for preserving the experience of playing alone, they fail to capture the dynamic social fabric that defined online multiplayer gaming in the 1990s and 2000s. The collapse of centralized matchmaking services, often due to corporate decisions or economic pressures, effectively severs the connection between players and their communities, rendering entire libraries of games unplayable [7]. To counteract this loss, the concept of an "Internet of Games" (IoG) emerges as a distributed, community-driven infrastructure designed to function as a "Wayback Machine" for multiplayer connectivity. This report details a comprehensive architectural blueprint for such a system, focusing on the creation of a protocol-agnostic foundation built upon proven technologies. The strategy prioritizes the development of a robust networking layer capable of hosting any legacy protocol, validated through targeted implementation for well-documented proving grounds like the GameSpy matchmaking service and IPX/SPX tunneling. The resulting framework is engineered for resilience, extensibility, and safety, leveraging modern systems languages like Rust to build a sustainable platform for the revival and continued play of classic multiplayer experiences.

The Interception and Redirection Layer: Gateway to Legacy Networks

The foundational challenge in reviving discontinued multiplayer games is the redirection of their network traffic away from defunct central servers and into a new, community-operated infrastructure. Many titles from the target era relied on hardcoded IP addresses or specific DNS hostnames to locate matchmaking, authentication, and master server services [12]. Modifying the original game binaries for every user is impractical, while altering global operating system settings can disrupt modern applications. The IoG architecture solves this by implementing a transparent interception and redirection layer

entirely within userspace, requiring no configuration changes on the client machine . This layer is composed of three primary technologies working in concert: a userspace TUN/TAP proxy for packet-level interception, a selective DNS override system for hostname redirection, and optional application-layer API hooking for handling particularly challenging networking implementations. Together, these components form the gateway through which all legacy game traffic flows, allowing the IoG infrastructure to inspect, modify, and reroute it according to the rules of the revived network.

The cornerstone of this interception mechanism is the use of virtual network interfaces, specifically TUN (Network Tunnel) and TAP (Network Tap) devices [11]. A TUN device operates at the Layer 3 (Network) level, presenting raw IP packets to the application, making it ideal for routing TCP and UDP traffic destined for legacy master servers [3]. A TAP device, operating at Layer 2 (Data Link), handles Ethernet frames, which is essential for bridging non-IP protocols like IPX/SPX [45]. The IoG framework utilizes Rust, leveraging its performance and safety features, to manage these interfaces. Crates such as `libpnet` and `socket2` provide the necessary low-level bindings to create and interact with TUN devices across different operating systems [11] [55]. The core of this functionality is encapsulated in a `wayback-core-net` crate, which provides a cross-platform abstraction for TUN device management . The initial implementation involves creating a minimal echo proxy, where a TUN device is configured, and packets read from it are immediately written back, establishing a functional virtual network link [57]. This serves as a diagnostic scaffold before routing logic is introduced. For enhanced performance, especially under heavy load, the architecture incorporates advanced I/O primitives. On Linux, leveraging `io_uring` via the `tokio-uring` crate enables zero-copy packet forwarding, minimizing data movement between kernel and userspace and reducing latency, which is critical for responsive gameplay [47].

To complement packet-level interception, a sophisticated DNS redirection strategy is essential. Many legacy games resolve domain names like `master.gamespy.com` to find matchmaking servers [7]. When these domains go offline, the IoG client must intercept these queries and return the address of a community-run replacement. Simply hijacking the system's DNS resolver is too blunt an instrument and risks breaking modern HTTPS traffic. The IoG architecture instead employs `hickory-dns`, a memory-safe and highly configurable DNS library written in Rust [55]. By constructing a custom stub resolver using `hickory-resolver`, the system can selectively poison DNS responses [18]. This resolver is configured to forward queries for modern, secure domains through standard resolvers while intercepting lookups for a predefined list of legacy game domains. For each intercepted query, it returns the IP address of a local IoG relay or a public bootstrap node, effectively rerouting the game's connection attempt. This approach ensures that

only legacy traffic is affected, leaving the rest of the user's internet experience intact. Furthermore, `hickory-dns` supports advanced record types like ANAME, which allows mapping a root domain (where CNAMEs are typically prohibited) to a dynamic IP address, providing a flexible mechanism for pointing multiple games to the same set of community nodes [2].

Despite the power of network and DNS-level interception, some legacy games employ unconventional networking stacks or aggressive anti-cheat mechanisms that bypass standard OS socket calls. In these cases, a final resort is application-layer hooking. The IoG framework plans to implement Windows-specific DLL injection and function detouring using crates like `detour-rs` and `windows-rs` [7]. This technique involves injecting a custom Dynamic-Link Library (DLL) directly into the address space of the game process. Once injected, the DLL can "hook" or replace the first few instructions of standard Winsock functions like `sendto`, `recvfrom`, and `gethostbyname` with jump instructions to its own proxy implementation [37]. This allows the IoG proxy to intercept network calls at the application layer, even if the OS-level transparent proxy has been bypassed. For example, a hook on `gethostbyname` could check if the game is attempting to resolve a known legacy master server hostname and immediately substitute the request with the IP address of the local IoG proxy, completely avoiding the need for any DNS modification. This method is powerful but carries significant complexity and risk, as improper implementation can easily crash the game or trigger anti-cheat heuristics. It is therefore considered a specialized tool for the most recalcitrant games, rather than a primary interception method. The combination of these three layers—TUN/TAP, DNS override, and application hooks—creates a multi-faceted gateway capable of capturing virtually any legacy game traffic and steering it onto the new, preserved network.

Component	Technology	Primary Function	Key Rust Crates
Packet Interception	Userspace TUN/TAP Device	Transparently intercepts and forwards IP packets (Layer 3) or Ethernet frames (Layer 2).	`tun`, `nix`, `tokio-uring` [11] [47]
Hostname Redirection	Selective DNS Resolver	Overrides DNS lookups for specific legacy domains to point to community-run masters.	`hickory-resolver`, `hickory-dns` [18] [55]
Application Hooking	API Detouring (Windows)	Intercepts Winsock API calls directly within the game's process memory.	`detour-rs`, `windows-rs` [7]

This interception and redirection layer forms the bedrock of the IoG architecture. By providing a transparent, configurable, and multi-layered approach to traffic manipulation, it creates the necessary entry point for the higher-level orchestration and execution layers, ensuring that legacy games can be seamlessly integrated into the revived multiplayer ecosystem.

The Decentralized Orchestration Layer: A Community-Controlled Brain

To ensure the longevity and resilience of the revived multiplayer experiences, the IoG framework must be built on a decentralized foundation that avoids the single points of failure that plagued the original services [21]. Centralized control is antithetical to the preservation ethos; a community-owned and operated network is far less susceptible to shutdowns due to corporate policy or financial constraints. The orchestration layer is the "brain" of the IoG, responsible for discovering active game servers, facilitating community-driven governance, and managing the overall state of the network without relying on a central authority. This is achieved primarily through two core technologies provided by the `libp2p` suite: the Kademlia Distributed Hash Table (DHT) for peer and content discovery, and the GossipSub protocol for scalable information dissemination and community voting [19] [35]. Together, these components create a self-organizing, fault-tolerant overlay network where every participant contributes to its stability and functionality.

The Kademlia DHT is the linchpin of the IoG's server discovery mechanism . In a traditional model, a client queries a central master server to get a list of available game servers. In the IoG's decentralized model, game servers act as "providers" and advertise their presence on the DHT, while clients act as "finders" who search for providers based on the game they want to play [28]. Each game is assigned a unique identifier, which is hashed (using SHA-256) to create a key in the DHT's keyspace. A game server that is hosting an instance of "Command & Conquer: Red Alert 2" would publish its connection information (a `Multiaddr`) under the key corresponding to the hash of "Red Alert 2". When a player opens the IoG lobby and searches for available servers, their client calculates the hash of "Red Alert 2" and contacts the `k` peers in the DHT that are "closest" to that key, where "closeness" is defined by a bitwise XOR metric on the 256-bit identifiers [20]. These peers, in turn, may not hold the answer themselves but will know other peers closer to the target key, gradually guiding the query toward the nodes that are storing the records for active game servers. The results are then returned to the client, providing a list of peers running the desired game [21]. This design, implemented using the `libp2p-kad` crate, is inherently resilient. With replication factors set to store each record on multiple nodes, the failure of individual peers or even entire geographic regions does not erase the information about available servers, ensuring the network remains functional [36].

While Kademlia excels at finding specific pieces of information (game servers), it is not optimized for broadcasting messages to a wide audience. For community-wide functions like voting for the "Game of the Week," proposing new server configurations, or sharing moderation reports, the IoG framework employs `libp2p-gossipsub` . GossipSub is a topic-based publish-subscribe protocol designed for efficient and robust dissemination of information across a large, loosely connected group of peers. When a node wants to announce something, it publishes a message to a specific topic (e.g., `"/iog/votes/weekly_featured_game"`) [12]. Peers interested in that topic subscribe to it and begin receiving messages. GossipSub uses a hybrid propagation model: when a peer receives a new message, it eagerly pushes the full message to a small, dense mesh of its nearest peers on that topic, and lazily sends only metadata (IHAVE tokens) to other peers, who then request the full message if interested (IWANT) [35]. This push-pull mechanism minimizes redundant bandwidth usage while ensuring that information propagates quickly throughout the network. Furthermore, GossipSub includes a scoring mechanism that rewards peers delivering high-quality data and penalizes those that are slow or malicious, helping to maintain the integrity of the shared information. This makes it an ideal substrate for a decentralized governance system, allowing the community to collectively make decisions without needing a central moderator or bulletin board.

A critical aspect of any decentralized governance system is preventing Sybil attacks, where a single entity creates numerous fake identities to gain disproportionate influence, such as bot-voting for their favorite game [31]. The IoG architecture explicitly acknowledges this threat and proposes a "Proof of Gameplay" (PoG) mechanism to mitigate it . While the specifics of PoG are left as an open research question, the principle is clear: earning the right to vote or participate in governance should require demonstrating genuine engagement with the games, not just creating accounts. This could involve requirements such as having played a certain number of hours, hosted a server, or contributed a verified protocol fix. Such a mechanism introduces a cost to malicious participation, whether it be computational (running a server), social (gaining reputation in the community), or temporal, thereby anchoring votes and governance actions to authentic human players. This combination of Kademlia for discovery and GossipSub for governance, fortified by a Sybil-resistant mechanism like PoG, creates a robust and community-centric orchestration layer. It transforms the IoG from a simple collection of relays into a living, self-sustaining network, guided by the collective will of its participants.

The Connectivity Bridge Layer: Spanning Modern Network Barriers

Legacy multiplayer games were conceived and developed in an era of direct IP connectivity, where dial-up modems and home LANs often provided stable, routable connections between peers. Today's internet landscape is vastly different, characterized by pervasive Network Address Translation (NAT) and firewalls that block unsolicited incoming connections, the very type required for classic peer-to-peer and client-server models [16]. A revived multiplayer game running on modern hardware behind a restrictive router cannot simply broadcast its presence and expect to be found. The IoG framework addresses this fundamental disconnect between old-world game networking and the new-world internet topology through a sophisticated Connectivity Bridge Layer. This layer acts as a universal translator and facilitator, employing modern protocols like WebRTC to traverse NATs and establish direct connections, and implementing legacy protocol tunneling (such as RFC 1234-compliant IPX over UDP) to allow non-TCP/IP games to run on an IP-based internet. This bridge is essential for restoring the full spectrum of classic gaming experiences, from LAN co-op to global online matches.

The premier technology for NAT traversal in the IoG framework is WebRTC, implemented via `str0m`, a "Sans I/O" WebRTC stack written in Rust . Unlike traditional networking libraries, `str0m` focuses solely on the WebRTC protocol logic—the state machines for ICE, DTLS, and SRTP—allowing the host application to plug in its own transport and threading logic [9]. This is ideal for the IoG's cross-platform goals. The IoG client acts as an Interactive Connectivity Establishment (ICE) agent, gathering a variety of "candidates" for its network address: "host" candidates (its private IP on the local network), "server-reflexive" candidates (the public IP and port seen by a STUN server), and potentially "relayed" candidates (if a TURN server is needed for highly restrictive NATs) [44]. It then exchanges these candidates with a peer and attempts to form direct connections. If a direct path is impossible, `str0m` can negotiate a connection through a relay node that has a publicly accessible address, ensuring connectivity even in the worst-case symmetric NAT scenarios [45]. Once a connection is established, it exposes a reliable stream (like a TCP socket) and unreliable datagram channels (like UDP sockets), perfectly emulating the communication primitives that legacy games expect. This allows the IoG to bridge the gap between a legacy PC game running on a desktop and a browser-based client, connecting them seamlessly over the public internet. For LAN-like experiences, WebRTC data channels can also be used to create a "virtual LAN" over the internet, allowing players to join a single session as if they were on the same physical network [54].

While WebRTC is a powerful tool for P2P connections, many classic games did not use TCP/IP at all. A significant portion of DOS-era gaming relied on Novell's Internetwork Packet Exchange (IPX) protocol, which was the default for early LANs [26]. The IoG framework must be able to tunnel this traffic to make those games playable again. The solution is to encapsulate IPX datagrams within UDP packets, following the specification outlined in RFC 1234 [39]. In this model, a UDP packet is constructed with a standard IP header, followed by a UDP header, and finally, the original IPX packet as the payload. The source and destination ports of the UDP header are standardized to UDP port 2644 (for the `travsoft-ipx-t` service) to identify the traffic [39]. The IoG implements this tunneling logic in a dedicated module, likely within the `wayback-core-net` crate. When a player runs a DOS game with an IPX driver, the game sends IPX packets to a local UDP listener. The IoG software on a community node receives these packets, wraps them in the appropriate UDP/IP headers, and forwards them over the internet to other nodes running the same tunneling service. Those nodes then strip off the outer UDP/IP headers and inject the clean IPX packets into their own virtual network stack, making the entire internet appear as a single, unified IPX network segment to the legacy games [15]. This capability is crucial for reviving the vast library of IPX-dependent titles.

Finally, the Connectivity Bridge Layer also encompasses the integration of modern web technologies to lower the barrier to entry. Instead of requiring players to download and install a full desktop client, the IoG can offer a lightweight lobby and basic gameplay experience directly in a web browser [5]. This is achieved by compiling the core networking logic to WebAssembly (WASM) using tools like `wasm-pack` or `trunk` [57]. This WASM module, running in a web worker, can handle the WebRTC signaling and packet processing. The browser's UI, built with a Rust-to-WASM frontend framework like Yew or Leptos, communicates with the WASM module to display server lists, send chat messages, and receive game updates [54]. Input from the browser's Gamepad API can be passed to the WASM netcode layer, which then forwards the inputs over the network to a desktop-hosted game server [3]. This browser-first approach allows players to discover games and join lobbies instantly, with the option to download the full desktop client later if they wish to host or play more demanding titles. The combination of WebRTC for universal connectivity, IPX tunneling for protocol compatibility, and WASM for web-native access ensures that the IoG framework can successfully span the chasm between legacy game networking and the complexities of the modern internet.

The Execution and Sandboxing Layer: Safe and Flexible Protocol Handling

The IoG framework's ability to be protocol-agnostic depends critically on its capacity to handle a vast and varied array of game-specific networking behaviors without embedding that logic directly into the core infrastructure. Every game from the 1990s and 2000s had its own unique way of structuring packets, negotiating connections, and maintaining state. Hardcoding handlers for hundreds of these protocols would defeat the purpose of building a general-purpose platform. The solution lies in a pluggable, sandboxed execution layer that allows community members to write and deploy protocol handlers as modular extensions. This layer provides two primary sandboxing technologies: Lua for lightweight, scriptable logic and WebAssembly (WASM) for high-performance, compiled code. By offering both options, the IoG can accommodate everything from simple packet-mutation scripts to computationally intensive protocol decoders, all while ensuring that untrusted or poorly written code cannot compromise the stability or security of the core IoG service .

Lua is selected as the primary scripting language for community-provided protocol handlers due to its minimal footprint, ease of embedding, and widespread use in game development [33]. The IoG framework will integrate a Lua virtual machine, likely using either `mlua` for bindings to a standard Lua interpreter or the `piccolo` crate for a pure-Rust, stackless implementation [9]. The stackless nature of `piccolo` is particularly advantageous for a high-concurrency networking application, as it allows Lua coroutines to be suspended and resumed without consuming system stack space, preventing stack overflow vulnerabilities and enabling fine-grained control over execution [6]. The architecture defines a clear contract between the IoG relay (`wayback-proxy`) and the Lua handler. When a packet is received, the relay invokes a specific Lua function (e.g., `on_packet(data)`), passing the raw packet bytes. The Lua script can then parse, modify, or drop the packet as needed and return the processed bytes. For example, a Lua script for a GameSpy-enabled game might intercept a server list request, rewrite the destination IP to point to an IoG master server, and then pass the modified packet along. All interaction with the outside world (e.g., logging, accessing configuration) must be mediated through a carefully crafted, restricted API exposed by the Rust host, preventing the script from performing dangerous operations like arbitrary filesystem access or network calls [10].

For more demanding tasks, such as parsing complex binary formats or implementing computationally expensive cryptographic algorithms for authentication, Lua's performance may be insufficient. This is where WebAssembly (WASM) becomes

invaluable. WASM provides a portable, high-performance compilation target that runs inside a secure sandbox [57]. The IoG framework will embed a WASM runtime, such as `wasmtime`, to execute protocol handlers compiled from languages like Rust or C++ [9]. Like the Lua interface, a WASM module will expose a simple function signature (e.g., `export fn handle_packet(buf_ptr: *const u8, buf_len: usize) -> PacketResult`) that the IoG relay can call. The benefits of this approach are significant. First, it offers near-native performance, suitable for tasks that would otherwise bottleneck the proxy. Second, it provides a much stronger security guarantee. WASM runtimes enforce strict memory isolation, and `wasmtime` includes a "fuel" mechanism that limits the number of CPU cycles a module can consume, preventing denial-of-service attacks from infinite loops or inefficient code [10]. Finally, it democratizes contribution by allowing developers to use any language that compiles to WASM, lowering the barrier to entry compared to writing native C++ plugins. The IoG proxy will act as a plugin loader, discovering `.so` (Linux) or `.dll` (Windows) files containing either Lua modules or WASM bytecode, loading them, and invoking their exported functions as needed. This dual-sandboxing strategy provides a flexible, secure, and performant extension system, empowering the community to drive the platform's growth by adding support for new games and fixing existing ones.

Sandbox Technology	Primary Use Case	Performance	Security Model	Example Crate(s)
Lua (Piccolo)	Lightweight scripting, packet mutation, declarative rule definition.	Moderate	Restricted FFI, custom allocator, instruction counting hooks.	`piccolo`, `mlua` [6] [9]
WebAssembly (Wasmtime)	High-performance decoding, complex crypto, computationally intensive logic.	High (near-native)	Memory-safe execution, CPU cycle limiting ("fuel").	`wasmtime`, `wasmer` [9] [10]

This modular execution layer is the key to the IoG's long-term viability. By abstracting protocol-specific logic into external, sandboxed modules, the core infrastructure remains lean, stable, and focused on its primary responsibilities of networking and orchestration. The community becomes the engine of expansion, continuously enriching the platform's capabilities by developing and sharing new protocol handlers, ensuring that the IoG can adapt and grow alongside the ever-expanding universe of classic multiplayer games.

AI-Assisted Protocols and On-Device Intelligence

Beyond its core networking functions, the IoG framework ambitiously integrates on-device artificial intelligence to augment the preservation effort itself. The sheer volume of

discontinued game services makes the manual reverse engineering of every protocol an insurmountable task for any community. The IoG architecture addresses this challenge by incorporating a local LLM inference engine, positioning AI as a collaborative partner for protocol archaeology, user support, and system maintenance . This AI-assisted layer is designed to operate on the user's local machine, ensuring privacy and responsiveness while providing powerful assistance in understanding and reconstructing the complex networking logic of legacy games. The AI's capabilities range from analyzing decompiled code to suggesting packet structures and acting as a real-time troubleshooting guide for end-users, fundamentally changing the workflow for preserving multiplayer history.

One of the most innovative applications of AI in the IoG is its use as a reverse engineering sidekick. Researchers and hobbyists can feed decompiled function disassembly or raw packet captures (`.pcap` files) into a locally running quantized LLM, such as one powered by the `mistral.rs` platform [7] [9]. The AI can then analyze this data to identify common patterns, suggest meaningful variable names, and explain the high-level intent of the code in plain language [49]. For example, when faced with a function responsible for generating a "seckey" for GameSpy authentication, feeding its decompiled body to the LLM can help identify the use of hardcoded game-specific keys and the structure of the challenge-response algorithm, drastically accelerating the process of building a functional master server replacement [7]. This lowers the barrier to entry for protocol recovery, enabling contributors with less expertise in low-level binary analysis to make meaningful contributions. The architecture treats the LLM as an API that takes structured data (decompiled code, hex dumps) and returns structured suggestions (packet format definitions, potential algorithm steps), which can then be verified and refined by humans. This iterative loop of AI suggestion and human verification is a core tenet of the project's methodology.

In addition to aiding developers, the AI, referred to as "AI-Chat," serves as a direct support tool for the end-user community. Players encountering issues with connectivity, NAT configuration, or game-specific quirks can interact with the AI chat interface to receive personalized, context-aware troubleshooting advice [22]. For instance, a user struggling to connect to a *Command & Conquer* server behind a restrictive firewall could ask, "AI, how do I get RA2 working behind my NAT?" The AI, equipped with a Retrieval-Augmented Generation (RAG) pipeline that draws from a knowledge base of documentation and past user discussions, could provide step-by-step instructions, including checking firewall settings or configuring UPnP [32]. This real-time, conversational support model is far more intuitive than searching through static forums or wikis. However, this integration requires careful consideration of security and privacy. The system must be designed to prevent prompt injection attacks, where a malicious

actor in a game chat attempts to trick the AI into revealing sensitive information like a host's IP address or executing unintended commands [32]. This necessitates a highly constrained prompt template and careful filtering of user inputs. Privacy-preserving techniques, such as federated learning for sentiment analysis, can be employed to train models on community behavior without centralizing raw chat logs [22].

The AI layer also extends to system maintenance and optimization. The IoG framework can leverage eBPF (Extended Berkeley Packet Filter) on Linux, via the `aya-rs` crate, or the `sysinfo` crate on other platforms, to gather detailed performance metrics from the host machine [3]. This data can inform the AI about the current load, available resources, and power status. The AI can then make intelligent recommendations, such as hibernating idle game servers to save electricity or gracefully downgrading the quality of a stream to prevent a player's machine from overheating . Furthermore, the AI can assist in automating the generation of protocol handler stubs. By analyzing a `.pcap` file of unknown traffic from a new game, the AI can propose a tentative Rust struct definition for the packet's fields, which a developer can then refine. This automated code generation, grounded in AI-generated hypotheses, further accelerates the process of adding support for new titles. The strategic placement of the AI as an on-device, privacy-conscious assistant that augments human intelligence rather than replacing it is a defining characteristic of the IoG architecture, promising to make the preservation of classic multiplayer gaming more accessible, efficient, and sustainable.

System Architecture and Implementation Roadmap

The successful realization of the Internet of Games framework hinges on a coherent system architecture and a pragmatic implementation roadmap. The design prioritizes modularity, safety, and a clear separation of concerns, mirroring the successful patterns established in related projects like Nintendoor64/GAMEMODE.ai . This is achieved by organizing the codebase into a set of focused Rust crates, each responsible for a specific subsystem, and enforcing a "contract-first" paradigm where all interactions are governed by typed Rust structs and their corresponding JSON schemas [18]. This approach ensures that the various components—from the low-level TUN proxy to the high-level Lua scripting engine—are interoperable and that the entire system can be reasoned about and managed by both developers and the envisioned AI-orchestrator. The implementation roadmap follows a vertical slice model, starting with a complete, end-to-end flow for a single, well-understood game to validate the core architecture before expanding to support additional protocols and features.

The proposed architecture is structured as a Rust workspace, with each major component residing in its own crate. This modularity is crucial for independent development, testing, and future reuse. Based on the research goal and conversation history, the key crates include:

- `wayback-core-net`: Manages the TUN/TAP device, transparent proxying, and low-level packet forwarding. It will wrap `libpnet` and `socket2` for cross-platform packet I/O [3] [11].
- `wayback-detect`: Implements a packet fingerprinting engine inspired by `rscap` to identify game engines from the initial bytes of a handshake, using a YAML or TOML-defined signatures file [7].
- `wayback-proxy`: Acts as the central orchestrator, routing packets to the appropriate game session, loading and managing Lua/WASM plugins, and enforcing circuit breakers for stability .
- `wayback-scripting`: Provides the safe sandboxed environments for Lua (`piccolo`) and WASM (`wasmtime`), exposing a restricted API to scripts for interacting with the network and system [9] [10].
- `wayback-dht`: Wraps the `libp2p` Kademlia and GossipSub implementations to provide decentralized discovery and community voting services [19].
- `wayback-webrtc-bridge`: Integrates `str0m` to provide WebRTC-based NAT traversal and a bridge to browser-based clients via WASM [9].
- `wayback-llm`: Houses the logic for the on-device AI, including the RAG pipeline for protocol recovery and the chat interface for user support [7].

This crate-based structure provides a clear ownership model and dependency graph, making the project manageable and understandable.

The "contract-first" paradigm is enforced through the extensive use of Rust's type system and the `schemars` crate to automatically generate JSON Schema definitions for all important data structures [18]. For example, the structure of a GameSpy heartbeat packet would be defined in a shared `iog-protocol-model` crate as a Rust struct:

```
// In iog-protocol-model crate
use schemars::JsonSchema;
use serde::{Deserialize, Serialize};

#[derive(Debug, Serialize, Deserialize, JsonSchema)]
pub struct GameSpyHeartbeat {
    pub query_port: u16,
    pub gamename: String,
```

```
    pub state_changed: u8,
    // ... other fields
}
```

These schemas become the official contract for how different parts of the system communicate. The `wayback-proxy` knows the exact structure of the packet to pass to the `wayback-scripting` engine. A Lua script author interacts with this schema to know what data they can access. And the AI-Chat can use the schema to understand the packet's meaning and generate appropriate modifications. This eliminates ambiguity and ensures that changes to protocols or internal APIs are made safely and consistently across the entire system.

The implementation roadmap is designed to deliver value incrementally, following a vertical slice pattern focused initially on the GameSpy proving ground . 1. **Phase 1: Core Interception and Echo Proxy.** Implement the `wayback-core-net` crate to create a functional TUN device and establish a simple echo proxy, as demonstrated in the preliminary code sample [57]. This validates the lowest level of packet interception. 2. **Phase 2: DNS Override and Basic Master Server.** Develop the `wayback-dns` crate using `hickory-dns` to redirect a specific legacy domain (e.g., `master.gamespy.com`). Simultaneously, build a basic TCP/UDP server in a new `wayback-master-gs` crate that can listen for GameSpy heartbeats and respond with a static server list [7]. 3. **Phase 3: End-to-End Flow for One Game.** Connect the components from Phases 1 and 2. Configure the IoG client to intercept traffic for a specific GameSpy title (e.g., *Command & Conquer*), redirect it to the local master server, and have the master server correctly handle the game's specific handshake and list encryption. This creates a fully functional, albeit basic, revival of the matchmaking for one game. 4. **Phase 4: Integrate Decentralization and Scripting.** Add the `wayback-dht` crate to have the IoG master server announce itself on the Kademlia DHT, making it discoverable by other nodes. Then, introduce the `wayback-scripting` crate and write a Lua script to perform a simple modification, like swapping a port number, demonstrating the extensibility of the system. 5. **Phase 5: Expand to Additional Proving Grounds and Features.** Once the GameSpy path is solid, add support for IPX/SPX tunneling to revive LAN-only DOS games. Integrate the WebRTC bridge to enable browser-based access. Finally, develop the AI-assisted features for protocol recovery and user support.

This phased approach mitigates risk by focusing on a small, achievable goal first. It produces tangible results quickly, builds confidence in the core architecture, and provides a stable foundation upon which to build more complex features. By combining a modular, crate-based architecture with a contract-first design philosophy and a

pragmatic, incremental implementation roadmap, the IoG framework is positioned to successfully navigate the immense technical challenges of preserving the social history of multiplayer gaming.

Reference

1. ZTNA support for UDP traffic | FortiGate / FortiOS 7.6.0 https://docs.fortinet.com/document/fortigate/7.6.0/new-features/843330/ztna-support-for-udp-traffic
2. File: CHANGELOG - Debian Sources https://sources.debian.org/src/nmap/7.40-1/CHANGELOG
3. Libp2p Rust tutorial - Stack Overflow https://stackoverflow.com/questions/79922024/libp2p-rust-tutorial
4. Unable to parse string "ACTL6AS5" at position 0 - Stack Overflow https://stackoverflow.com/questions/70362047/unable-to-parse-string-actl6as5-at-position-0
5. udp-proxy · GitHub Topics https://www.github-zh.com/topics/udp-proxy
6. Creating a secure Lua sandbox..? - Stack Overflow https://stackoverflow.com/questions/34388285/creating-a-secure-lua-sandbox
7. MOOs - an overview | ScienceDirect Topics https://www.sciencedirect.com/topics/computer-science/moos
8. Quake Series | PDF | Video Game Development - Scribd https://www.scribd.com/document/70127626/Quake-Series
9. Data-Centric Serverless Computing with LambdaStore - MDPI https://www.mdpi.com/2674-113X/5/1/5
10. [PDF] WebAssembly as an Abstraction for Secure and Efficient Computing ... https://hal.science/tel-05544265v1/file/JamesMenetrey_Thesis_00003199.pdf
11. TPROXY is not redirecting all traffic to a specified port - Server Fault https://serverfault.com/questions/1143600/tproxy-is-not-redirecting-all-traffic-to-a-specified-port
12. (PDF) Investigating Spyware in Peer-to-Peer Tools - ResearchGate https://www.researchgate.net/publication/228759206_Investigating_Spyware_in_Peer-to-Peer_Tools
13. (PDF) Multiplayer Game Programming - Academia.edu https://www.academia.edu/43559397/Multiplayer_Game_Programming

14. [PDF] Cisco Firepower Application Detector Reference - VDB 304 https://www.cisco.com/c/en/us/td/docs/security/firepower/Application_Detectors/vdb_304/cisco_firepower_application_detector_reference_304.pdf
15. Online Games Traffic Multiplexing: Analysis and Effect in Access ... https://www.researchgate.net/publication/263807609_Online_Games_Traffic_Multiplexing_Analysis_and_Effect_in_Access_Networks
16. An Empirical Study on Remote DoS Attacks against NAT Networks https://arxiv.org/html/2410.21984v1
17. Cryptographic Puzzles and Game Theory against DoS and DDoS ... https://www.academia.edu/892474/Cryptographic_Puzzles_and_Game_Theory_against_DoS_and_DDoS_attacks_in_Networks
18. Fast request sending in Rust - reqwest - Stack Overflow https://stackoverflow.com/questions/79396900/fast-request-sending-in-rust
19. S/Kademlia: A practicable approach towards secure key-based routing https://www.researchgate.net/publication/4319659_SKademlia_A_practicable_approach_towards_secure_key-based_routing
20. Generalized Distance Metric for Various DHT Routing Algorithms in ... https://arxiv.org/html/2303.13965v2
21. [PDF] From key-based to content-based routing: system interconnection ... https://theses.hal.science/tel-00875653/document
22. 从ZLibrary入口看数字资源分发架构：类比CDN、P2P、IPFS与去 ... https://gitcode.csdn.net/69def94a0a2f6a37c59fca73.html
23. The Design and Implementation of a Graph-Based P2P Data ... - MDPI https://www.mdpi.com/1999-5903/18/1/9
24. Exploring the design space of privacy-enhanced content discovery ... https://www.sciencedirect.com/science/article/pii/S014036642400029X
25. Lattica: A Decentralized Cross-NAT Communication Framework for ... https://arxiv.org/html/2510.00183v1
26. Game Design Foundations | PDF - Scribd https://www.scribd.com/document/271300699/Game-Design-Foundations
27. 333333 23135851162 the 13151942776 of 12997637966 ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt
28. Enhancing Crowd-Sourced Video Sharing through P2P-Assisted ... https://www.mdpi.com/2079-9292/13/7/1270

29. In simple terms, how does a BitTorrent client initially discover peers ... https://superuser.com/questions/592238/in-simple-terms-how-does-a-bittorrent-client-initially-discover-peers-using-dht
30. An Architectural Survey Toward a GenAI-Driven Future - IEEE Xplore https://ieeexplore.ieee.org/iel8/6287639/10820123/11296799.pdf
31. [PDF] Service discovery and selection in IoT: A survey and a taxonomy - HAL https://hal.science/hal-04431300v1/file/S1084804521003167.pdf
32. [PDF] An Architectural Survey Toward a GenAI-Driven Future - arXiv https://arxiv.org/pdf/2512.07638
33. Ring: A Lightweight and Versatile Cross-Platform Dynamic ... - MDPI https://www.mdpi.com/2079-9292/13/23/4627
34. Ubuntu 24.04 LTS release notes https://documentation.ubuntu.com/release-notes/24.04/
35. Network-level Censorship Attacks in the InterPlanetary File System https://arxiv.org/html/2509.06626v1
36. Release Notes — py-libp2p 0.6.0 documentation https://py-libp2p.readthedocs.io/en/stable/release_notes.html
37. [PDF] Cisco Firepower Application Detector Reference - VDB 342 https://www.cisco.com/c/en/us/td/docs/security/firepower/Application_Detectors/vdb-342/cisco_firepower_application_detector_reference_342.pdf
38. https - cs.Princeton https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt
39. TCP/UDP等所有端口号列表及其服务 - 撒欢 - 博客园 https://www.cnblogs.com/frisk/p/12459446.html
40. socket默认端口号(socket和端口的关系) - 腾讯云 https://cloud.tencent.com/developer/article/2063311
41. Linux端口号对应服务文件 - 阿里云开发者社区 https://developer.aliyun.com/article/888860
42. IP Addressing Configuration Guide, Cisco IOS XE 17.x https://www.cisco.com/c/en/us/td/docs/routers/ios/config/17-x/ip-addressing/b-ip-addressing/m_iadnat-stateful-nat64.html
43. 10-缩略语-新华三集团-H3C https://www.h3c.com/cn/d_201908/1218977_30005_0.htm
44. STUN - Simple traversal of UDP through network address translators ... https://www.researchgate.net/publication/242504546_STUN_-_Simple_traversal_of_UDP_through_network_address_translators_IETF-Draft

45. Tunnel Management - an overview | ScienceDirect Topics https://www.sciencedirect.com/topics/computer-science/tunnel-management
46. [PDF] Dell Networking W-Series ArubaOS 6.5.x Command-Line Interface https://dl.dell.com/manuals/common/instant-6431-42-cli-guide.pdf
47. sle-micro-6-0-byos-v20250630-x86-64 Package Source Changes https://publiccloudimagechangeinfo.suse.com/google/sle-micro-6-0-byos-v20250630-x86-64/package_changelogs.html
48. cmnt_vocab.txt - CMU School of Computer Science https://www.cs.cmu.edu/~ark/blog-data/data/blog_data_v1_0/dk/hbc_data/data/cmnt_vocab.txt
49. [PDF] Distributed Deep Learning In Open Collaborations - arXiv https://arxiv.org/pdf/2106.10207
50. Technical Reports | Department of Computer Science, Columbia ... https://www.cs.columbia.edu/technical-reports/
51. Reliable User Datagram Protocol as a Solution to Latencies in ... https://www.mdpi.com/2079-9292/7/11/295
52. (PDF) Reliable User Datagram Protocol as a Solution to Latencies in ... https://www.researchgate.net/publication/328739841_Reliable_User_Datagram_Protocol_as_a_Solution_to_Latencies_in_Network_Games
53. 5G SA Core Network Architecture: A Cloud-Native, Modular Design https://www.linkedin.com/posts/amit-kumar-singh-307875297_5g-sa-core-network-architecture-5g-standalone-activity-7371424019566354432-VFRr
54. File: CHANGELOG - Debian Sources https://sources.debian.org/src/nmap/7.91+dfsg1+really7.80+dfsg1-2/CHANGELOG/
55. Debian -- Software Packages in "trixie", Subsection rust https://packages.debian.org/stable/rust/
56. Capstone-Assignment - RPubs https://rpubs.com/BushraT/1075426
57. Rust vs JavaScript & TypeScript: Performance and WebAssembly https://blog.jetbrains.com/rust/2026/01/27/rust-vs-javascript-typescript/
58. [PDF] Exploiting player behavior in distributed architectures for online games https://theses.hal.science/tel-00931865v1/file/Thesis.pdf
59. What happened after DNS TTL expired in intermediate Name Server? https://stackoverflow.com/questions/51353779/what-happened-after-dns-ttl-expired-in-intermediate-name-server
60. Module 8 Review Quiz Flashcards - Quizlet https://quizlet.com/589205398/module-8-review-quiz-flash-cards/

61. Allwords Withdigits | PDF | Breastfeeding | Breast Milk - Scribd https://www.scribd.com/document/574115973/allwords-withdigits